

Evolution of

Neural Networks

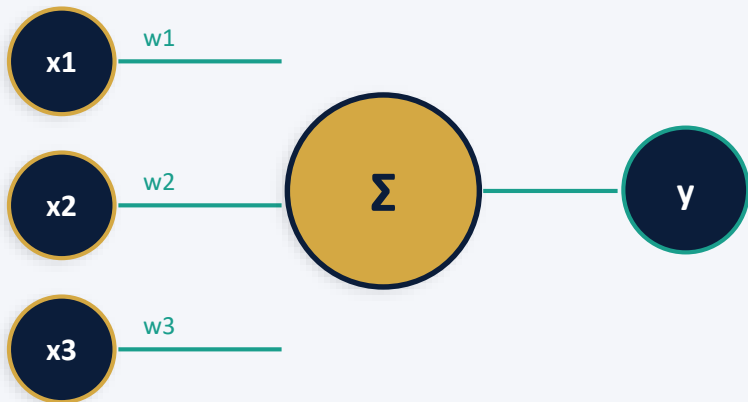
From McCulloch & Pitts (1943) to the Modern AI Era

1943 → 1958 → 1969 → 1982 → 1986 → 1997 → 2006 → 2012 → 2017 → Today

1943

McCulloch & Pitts

The First Mathematical Model of a Neuron



Warren McCulloch & Walter Pitts

Published "A Logical Calculus of the Ideas Immanent in Nervous Activity"

Binary Threshold Model

Neurons fire (1) or stay silent (0) based on weighted sum of inputs exceeding a threshold

Logical Operations

Showed the model could compute AND, OR, NOT — the basis of all computation

Foundation Stone

First proof that neural computation is mathematically formalizable — ignited the entire field

The MP Neuron — How It Actually Works

Mathematical formalization of biological neurons

Step 1 - Weighted Sum

$$a = \text{Sum}(w_i * x_i) = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n$$

Each input x_i is multiplied by its synaptic weight w_i and summed together

Step 2 - Threshold Check

$$y = 1 \text{ if } a \geq \text{threshold, else } y = 0$$

Output fires (1) only if the weighted sum meets or exceeds threshold θ

Step 3 - Logical Power

$$\text{AND: } \theta=2, w=1 \quad | \quad \text{OR: } \theta=1, w=1 \quad | \quad \text{NOT: } \theta=0, w=-1$$

By choosing weights and threshold, any Boolean function can be computed

Biological Neuron

MP Neuron

Dendrites receive signals

Inputs $x_1, x_2 \dots x_n$

Synaptic strength

Weights $w_1, w_2 \dots w_n$

Cell body integrates

Summation ($\text{Sum } w_i * x_i$)

Action potential fires

Threshold activation

Axon transmits output

Binary output y : 0 or 1

1958

Frank Rosenblatt

The Perceptron — First Learning Algorithm

1958

Year Invented

Mark I

Perceptron Machine

1st

Learning Network

- ▶ Introduced the Perceptron — a single-layer network that could learn from examples
- ▶ Implemented on the Mark I Perceptron machine — a real hardware neural network
- ▶ Used the Perceptron Learning Rule: adjust weights when the prediction is wrong
- ▶ Demonstrated pattern recognition on images — classifying shapes without explicit programming
- ▶ Rosenblatt's bold claim: 'The perceptron is the embryo of an electronic computer that will be able to walk, talk, see, write, reproduce itself' (1958)

The Perceptron Learning Rule

The world's first algorithm that could learn from its own mistakes

Perceptron Update Rule

$$w(\text{new}) = w(\text{old}) + \text{learning_rate} \times (\text{target} - \text{output}) \times \text{input}$$

where learning rate (η) is typically 0.01 to 0.1

Worked Example (learning rate = 0.1)

Input: $x=[1, 0]$, $\text{target}=1$, $w=[0.2, 0.5]$, $\text{threshold}=0.6$

Compute: $a = 1 \times 0.2 + 0 \times 0.5 = 0.2 \Rightarrow \text{output}=0$ ($0.2 < 0.6$) WRONG

Error: $\text{error} = \text{target} - \text{output} = 1 - 0 = 1$

Update w_1 : $w_1 = 0.2 + 0.1 \times 1 \times 1 = 0.30$

Update w_2 : $w_2 = 0.5 + 0.1 \times 1 \times 0 = 0.50$ (no change, $x_2=0$)

Iterate: Repeat until all examples classified correctly

Convergence Theorem

If data IS linearly separable, the perceptron WILL converge to a perfect solution — mathematically guaranteed by Rosenblatt.

No Backpropagation

Weights update at the output layer only — a local rule. No chain rule required. Simple and elegant.

Why It Failed on XOR

XOR is NOT linearly separable — no single straight line can divide the classes. Single layer cannot solve it.

Lasting Legacy

The delta rule (error $\times$ input) is the foundation of gradient descent used in every modern neural network today.

1969

Minsky & Papert

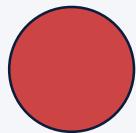
"Perceptrons" — The Book That Triggered the First AI Winter



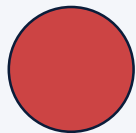
$(0,1) \rightarrow 1$



$(1,0) \rightarrow 1$



$(0,0) \rightarrow 0$



$(1,1) \rightarrow 0$

XOR — not linearly separable

Critical Flaw Exposed

Proved single-layer perceptrons CANNOT solve XOR — or any non-linearly separable problem

Funding Collapse

US & UK governments cut AI research funding; neural network research stagnated for ~15 years

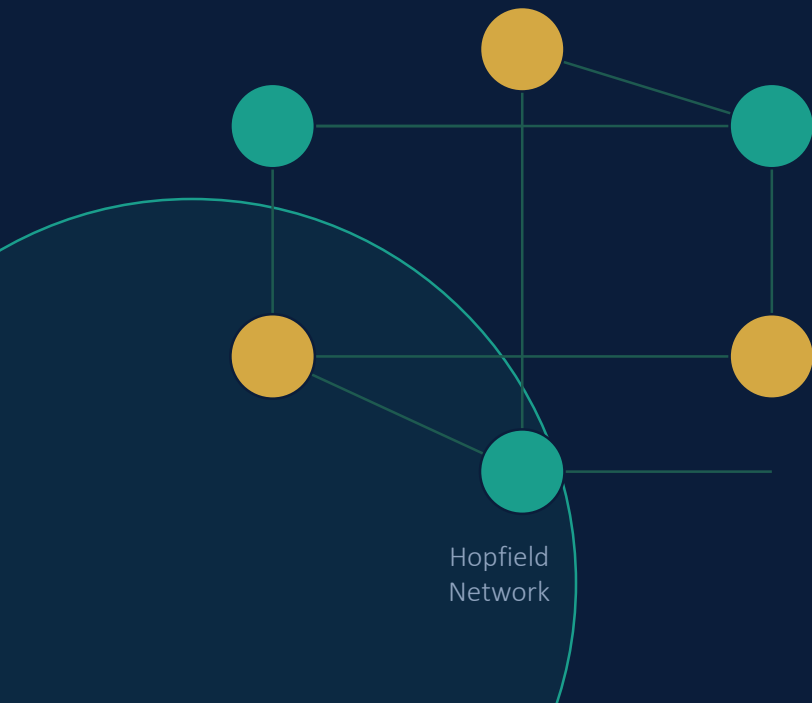
The Silver Lining

Their critique inadvertently defined what was needed: multi-layer networks with non-linear functions

1982

John Hopfield

Recurrent Networks & Associative Memory — Nobel Prize 2024



Hopfield
Network

Energy-Based Recurrent Network

Neurons are fully connected and update until reaching a stable low-energy state — like a ball settling in a valley

Associative Memory

Can store and retrieve patterns — give it a noisy/partial memory and it reconstructs the original

Physics-Inspired AI

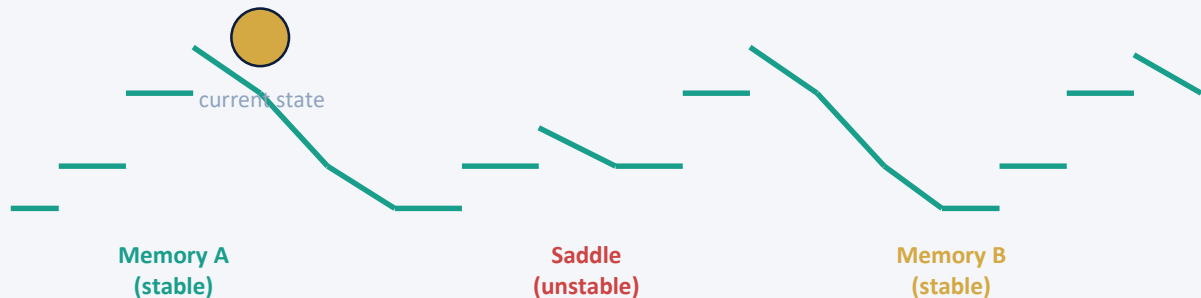
Borrowed energy minimization from statistical physics (spin glasses) — bridged two scientific worlds

Nobel Prize 2024

Awarded the Nobel Prize in Physics alongside Geoffrey Hinton for foundational discoveries in machine learning

Hopfield Networks — Memory as Energy Minimisation

Physics of spin glasses translated into associative memory



Network rolls downhill to stored memory attractors

Energy Function

$E = -1/2 \sum (w_{ij} * s_i * s_j)$. Minimized at stored memories (attractors)

Storage Capacity

Stores approximately $0.14N$ patterns reliably for N neurons

Auto-Association

Present partial or noisy input and network retrieves the complete stored pattern

Spin Glass Connection

Borrowed energy minimization from condensed matter physics — bridged two sciences

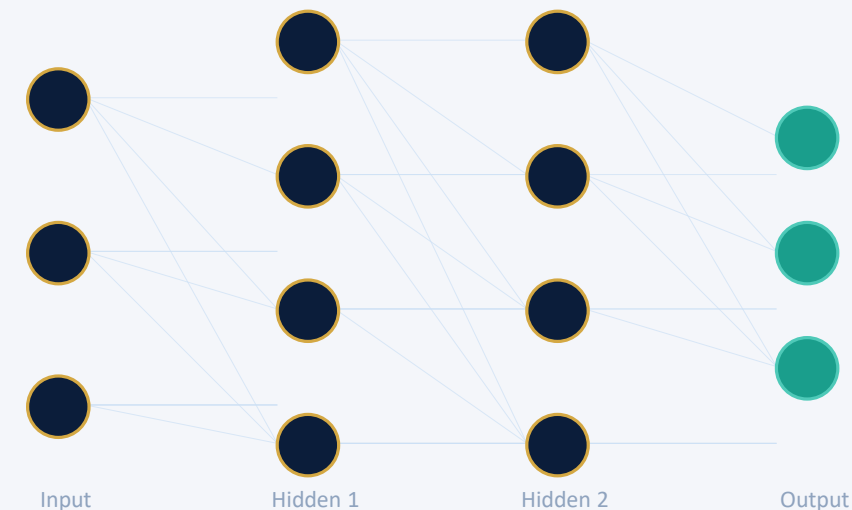
Modern Hopfield (2020)

Ramsauer et al.: exponentially more patterns stored — mathematically equivalent to Transformer attention

1986

Geoffrey Hinton

Backpropagation — The Algorithm That Made Deep Learning Possible



← Error gradient flows backward

Backpropagation Paper (1986)

Co-authored with Rumelhart & Williams — showed how gradients can train multi-layer networks

Chain Rule Applied

Error flows backward layer by layer; each weight updated proportionally to its contribution to the error

Distributed Representations

Features learned automatically — not hand-designed — by distributing knowledge across all weights

2006: Deep Belief Networks

Showed how to pre-train deep networks layer by layer — reignited deep learning research

Nobel Prize 2024

Awarded jointly with Hopfield for foundational contributions that enabled modern machine learning

Backpropagation — The Chain Rule in Action

How errors flow backward to update every weight in the network

1

Forward Pass

Input flows through layers. Each neuron computes weighted sum $z = \sum(w_i * x_i)$, then applies sigmoid activation $y = 1/(1+\exp(-z))$

2

Compute Loss

Compare output with target: $L = 0.5 * (y_{\text{hat}} - y_{\text{target}})^2$. Gradient: $dL/dy_{\text{hat}} = y_{\text{hat}} - y_{\text{target}}$

3

Backward Pass

Chain rule: $dL/dw = dL/dy_{\text{hat}} * dy/dz * dz/dw = \text{error} * \text{sigmoid_prime}(z) * x$

4

Weight Update

Gradient descent: $w_{\text{new}} = w_{\text{old}} - \text{eta} * dL/dw$. Repeat until loss is minimized across all examples.

The Vanishing Gradient Problem and Its Solution

Sigmoid derivative peaks at only 0.25 and shrinks to near zero for large inputs. In deep networks, multiplying many of these tiny values together means gradients vanish — deep layers learn nothing. The fix: ReLU activation ($\max(0, x)$, derivative = 1 for positive inputs) and Residual connections (ResNet, 2015) that allow gradients to flow directly through skip connections.

1989 – 1998

Breakthroughs: CNNs, LSTMs & Quiet Progress

Three landmark contributions that shaped modern AI

1989

Yann LeCun

Convolutional Neural Networks

- Applied backprop to image recognition
- Invented Conv layers: local filters that detect edges, shapes, textures
- LeNet recognized handwritten digits — deployed in US bank cheque readers by 1998

1997

Hochreiter & Schmidhuber

Long Short-Term Memory (LSTM)

- Standard RNNs forget long-range dependencies — vanishing gradient problem
- LSTM adds gating: input, forget, output gates control memory flow
- Enabled language translation, speech recognition, time series — used in Siri, Google Translate

1995–98

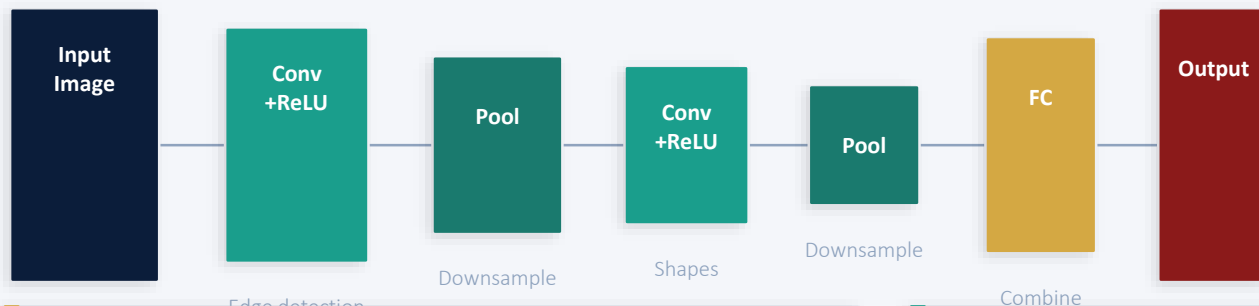
Cortes, Vapnik, Breiman

SVMs & Ensemble Methods

- Support Vector Machines dominated practical ML tasks
- Random Forests and Boosting provided strong competitors to NNs
- Neural networks suffered a second "winter" — SVMs won most benchmarks

Convolutional Neural Networks — Yann LeCun

Biologically inspired spatial feature detection — from pixels to concepts



Local Receptive Fields

Each neuron sees only a small patch — mimicking biological visual cortex neurons that respond to local edges and shapes

Pooling / Subsampling

Max pooling reduces spatial dimensions — provides translation invariance (recognises a digit wherever it appears)

Weight Sharing

Same filter applied across entire image — 95% fewer parameters vs fully connected; forces learning translation-invariant features

LeNet-5 Real Impact

Deployed in US Postal Service and banks — reading over 10% of all handwritten cheques in the USA by late 1990s

Long Short-Term Memory (LSTM)

Hochreiter & Schmidhuber — teaching networks to remember and forget selectively

**f**

Forget Gate

$$f = \text{sigmoid}(W_f * [h, x] + b_f)$$

Decides what to erase from cell state. Output near 0 = forget everything; near 1 = keep everything. Solves catastrophic forgetting in plain RNNs.

**i**

Input Gate

$$i = \text{sigmoid}(W_i * [h, x] + b_i), c \sim = \tanh(W_c * [h, x] + b_c)$$

Decides what new information to write into cell state. Gate selects which values matter; tanh creates candidate values between -1 and 1.

**o**

Output Gate

$$o = \text{sigmoid}(W_o * [h, x] + b_o), h = o * \tanh(C)$$

Decides which part of the cell state to expose as hidden state for the next timestep and for prediction output.

Cell State (C_t) — the memory highway running through all timesteps, only gently modified by gates — this is how LSTMs retain information over hundreds of timesteps

The AI Winters — What Killed Progress Twice

Understanding why the field collapsed — and why it came back stronger each time

First AI Winter

1969 – 1982

What killed it:

- X Minsky & Papert proved perceptrons cannot solve XOR (1969)
- X DARPA and UK government slashed all AI funding (1973)
- X Lighthill Report condemned AI as fundamentally oversold
- X No hardware capable of running useful networks existed

What revived it:

- V Hopfield energy-based networks showed new possibilities (1982)
- V Rumelhart PDP books revived distributed representations
- V Japan Fifth Generation project renewed international interest in AI

Second AI Winter

1987 – 2006

What killed it:

- X Lisp Machine hardware market collapsed — ecosystem gone
- X Expert systems proved brittle and extremely expensive to maintain
- X Support Vector Machines outperformed neural nets on benchmarks
- X Backprop too slow on available hardware for any useful depth

What revived it:

- V Hinton Deep Belief Networks showed deep training IS possible (2006)
- V LeCun and Bengio continued publishing quietly without fanfare
- V GPU hardware finally made large-scale training economically feasible

Hardware and Data — The Unsung Heroes

Algorithms alone were not enough — compute and data had to catch up first

GPU

GPU Revolution

- 2007: CUDA enables general-purpose GPU computing
- 2009: Raina et al. show GPUs train NNs 70x faster than CPUs
- 2012: AlexNet on GTX 580 — changed the entire field overnight
- 2016: NVIDIA Pascal P100 — purpose-built for deep learning
- 2020s: H100 / A100 — over 1000x faster than 2012 hardware
- 2024: Custom AI chips (TPU, Trainium) — billions invested

DATA

Data Explosion

- 2009: ImageNet — 14 million labelled images by Fei-Fei Li
- 2010: ImageNet Challenge (ILSVRC) drives annual benchmark
- Common Crawl: 250TB of web text available free to researchers
- Wikipedia + BooksCorpus powered BERT and early GPT
- Web-scale data (trillions of tokens) trains GPT-4 and Claude
- Synthetic data generation now augments real-world datasets

CLOUD

Cloud Compute

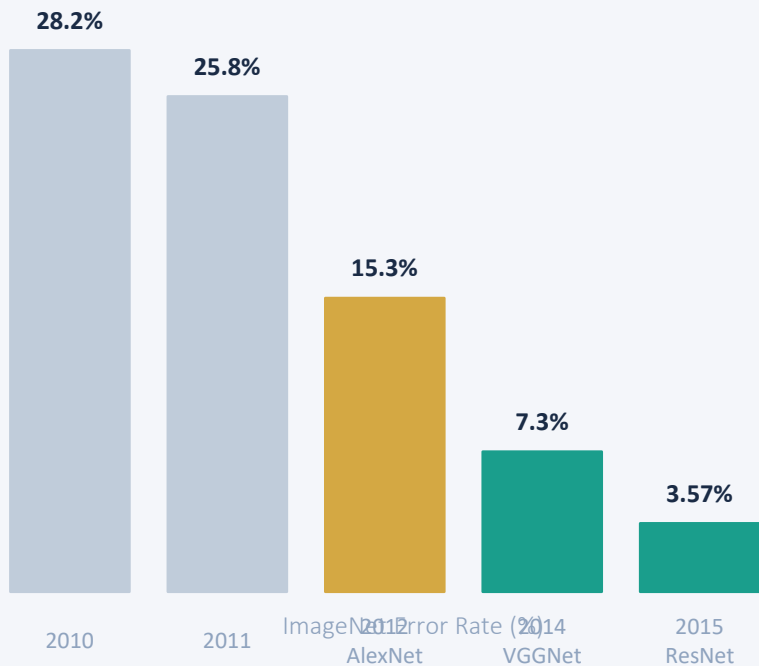
- 2006: AWS EC2 democratises large-scale on-demand compute
- 2017: Google TPU v1 — custom AI chip at 92 TOPS
- 2020: GPT-3 training cost approx 4.6M USD — feasible for labs
- 2023: H100 clusters with 10,000+ GPUs — industry standard
- 2024: Microsoft, Google, Amazon racing to build AI datacenters
- Estimated 100B USD in AI infrastructure investment per year

The deep learning revolution was a perfect storm: the RIGHT algorithms + the RIGHT hardware + the RIGHT data — all arriving simultaneously around

2012

Krizhevsky, Sutskever & Hinton

AlexNet — The Moment Deep Learning Won



8 Layers Deep

First practical very deep network — 5 conv layers + 3 fully connected — trained on 2 GPUs

ReLU Activation

Replaced sigmoid with Rectified Linear Units — 6x faster training, solved vanishing gradient

Dropout Regularisation

Randomly zeros neurons during training — prevents overfitting in large networks

GPU Training

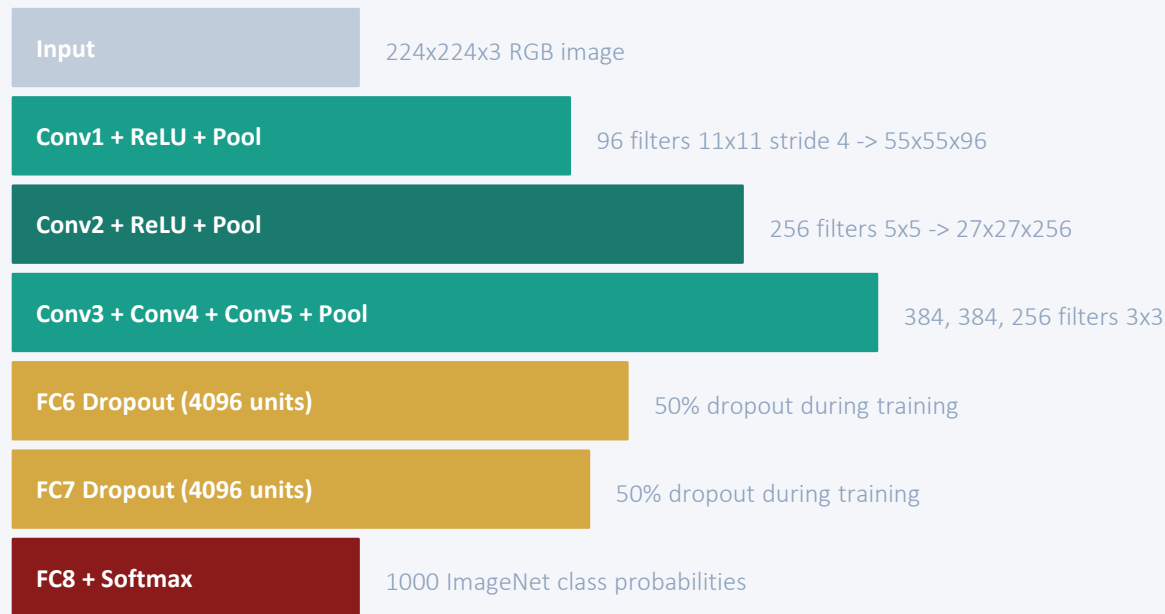
Used NVIDIA GTX 580 GPUs — proved that hardware acceleration was the missing key

15.3% Error → Revolution

Crushed 2nd place (26.2%) — overnight every lab switched from SVMs to deep learning

AlexNet — Why It Really Won ImageNet

Five innovations firing simultaneously — not one, but all five together caused the breakthrough



Key Innovations: ReLU | Dropout 0.5 | Data Augmentation | Dual-GPU training | Local Response Normalisation

2017 → Present

Transformers & the Modern AI Era

"Attention is All You Need" — Vaswani et al., Google 2017



Self-Attention Mechanism

Every token attends to every other token — parallelizable, captures long-range dependencies

Scale = Intelligence

More data + more parameters + more compute $\approx$ emergent capabilities

Foundation Models

Train once on everything, fine-tune for anything — DALL-E, GPT-4, Gemini, Claude

The Transformer Attention Mechanism Explained

Attention Is All You Need — Vaswani et al., Google Brain, 2017

Query (Q)

What am I looking for?

Key (K)

What do I advertise?

Value (V)

What do I actually return?

Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) * V$$

1. Dot Product $Q \cdot K^T$

Score how relevant each key is to the current query. High score = the token is useful context.

2. Scale by $1/\sqrt{d_k}$

Prevents softmax from saturating and losing gradient in high-dimensional spaces.

3. Softmax into weights

Convert raw scores to probabilities summing to 1 — top tokens get most of the attention weight.

4. Weighted sum of V

Retrieve information from values weighted by attention scores — the output carries context.

Multi-Head Attention: 8-16 attention heads run in parallel — each learns different relationships (syntax, coreference, semantics, position)

The Neural Network Family Tree

80+ years of ideas building on each other

1943 **McCulloch & Pitts**

Mathematical neuron model

1958 **Frank Rosenblatt**

Perceptron + Learning Rule

1969 **Minsky & Papert**

XOR problem → 1st AI Winter

1982 **John Hopfield**

Recurrent networks, energy minimization

1986 **Geoffrey Hinton et al.**

Backpropagation — trains multi-layer nets

1989–97 **LeCun / Hochreiter & Schmidhuber**

CNNs (LeNet) + LSTM

2006 **Hinton (Deep Belief Nets)**

Deep learning revival

2012 **Krizhevsky, Sutskever, Hinton**

AlexNet — GPU deep learning wins ImageNet

2017 **Vaswani et al. (Google)**

Transformer — Attention is All You Need

2022+ **OpenAI, Google, Anthropic...**

ChatGPT, Gemini, Claude — AI for everyone